

Análise Crítica

Testes Unitários e Cobertura de Código

Análise Crítica – Testes Unitários e Cobertura de Código

Este trabalho teve como objetivo **aplicar práticas de testes de software em um módulo simples desenvolvido em Python, responsável por operações relacionadas ao cálculo e análise de notas de alunos**. O módulo implementado contém três funções principais: cálculo da média de notas, verificação da situação do aluno (aprovado, recuperação ou reprovado) e identificação da maior nota em uma lista.

A escolha desse módulo se justifica por conter lógica de negócio clara e regras bem definidas, o que facilita a construção de testes representativos e a análise de qualidade do código. Apesar de sua simplicidade, o módulo apresenta cenários suficientes para aplicação de boas práticas de teste.

Foram implementados testes unitários utilizando a biblioteca padrão **unittest** da linguagem Python. Os testes foram estruturados de forma a cobrir diferentes tipos de cenários, incluindo **casos normais, casos de erro e casos de limite**. Nos cenários principais, foram verificados comportamentos esperados, como **o cálculo correto da média de uma lista de notas e a classificação adequada do aluno com base na média obtida**. Estes testes garantem que as funcionalidades básicas do sistema estão operando corretamente.

Além disso, foram considerados **casos de erro, como a tentativa de calcular a média ou identificar a maior nota em uma lista vazia**. Nessas situações, o sistema foi projetado para lançar exceções, e os testes confirmam que esses erros são tratados de forma adequada. Isso é importante para garantir a robustez do sistema e evitar comportamentos inesperados em tempo de execução.

Os testes também contemplaram **casos de limite, como valores de média exatamente iguais aos limites de aprovação e recuperação**. Esse tipo de teste é essencial, pois muitos erros em sistemas reais ocorrem justamente em condições de fronteira. A validação desses cenários contribui para aumentar a confiabilidade do código.

Para avaliar a qualidade dos testes, foi utilizada a ferramenta de cobertura de código **coverage**, que permite **identificar quais partes do código foram executadas durante a execução dos testes**. A cobertura obtida foi satisfatória, indicando que a maior parte das instruções do módulo foi efetivamente testada. Isso demonstra que os testes possuem boa abrangência e são capazes de detectar possíveis falhas nas funções implementadas.

Apesar dos resultados positivos, a análise crítica permitiu identificar **algumas limitações no projeto**. Uma das principais lacunas é **a ausência de validação de tipos de entrada**. Atualmente, o sistema assume que as notas fornecidas são sempre valores numéricos válidos,

o que pode não ser verdadeiro em um cenário real. A inclusão de validações adicionais tornaria o sistema mais robusto.

Outra limitação observada é a **falta de testes para valores extremos, como notas negativas ou valores muito elevados**. Embora esses casos possam não ocorrer em condições normais, sua consideração é importante para garantir maior confiabilidade e prevenir erros inesperados.

Além disso, **o sistema não contempla cenários mais complexos, como listas muito grandes ou integração com outras partes de um sistema maior**. Em um contexto real, seria importante expandir os testes para incluir esses aspectos, bem como adotar práticas como testes automatizados em pipelines de integração contínua.

Como **melhorias futuras**, recomenda-se a **implementação de validações de entrada, a ampliação dos testes para cobrir cenários extremos e a refatoração do código para torná-lo mais modular e escalável**. Também seria interessante **utilizar frameworks de teste mais avançados**, como **pytest**, que oferecem maior flexibilidade e recursos adicionais.

Em conclusão, o trabalho permitiu aplicar conceitos fundamentais de testes de software e evidenciou a importância da validação sistemática do código. A utilização de testes unitários e análise de cobertura contribui significativamente para a melhoria da qualidade do software, reduzindo riscos e aumentando a confiabilidade das aplicações desenvolvidas.